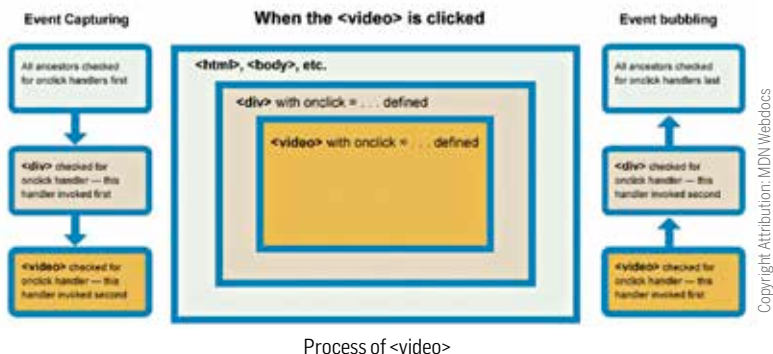


In the capturing phase:

- If the element's outermost ancestor (**<html>**) has an **onclick** event handler recorded for the capture process, the browser can execute it.
- Then it goes on to the next entity within **<html>** and repeats the process, then the next, and so on before it reaches the final selected element.

In the bubbling process, the exact opposite occurs:

- The browser tests to see if the selected entity has an **onclick** event handler for the bubbling process, and if it does, it executes it.
- Then it repeats the process for the next immediate ancestor node, then the next, and so on before it hits the **<html>** element.



Both event handles are registered for the bubbling process by design in modern browsers. When you pick the video in our current case, the event bubbles outwards from the **<video>** element to the **<html>** element. On the way, you'll encounter:

It locates and executes the **file.onclick...** handler, enabling the video to begin playing.

The **videoBox** is then discovered, **onclick...** handler and operates it, then the video is hidden as well.

StopPropagation()

This is a very troublesome action, but there is a solution! The basic Event object has a feature named **stopPropagation()** that, when called on a handler's event object, causes the first handler to be executed but the event to not propagate further up the chain, resulting in no further handlers being executed.